

# Objetivo da Migração e Roteiro Sugerido

Este material de referência detalha a justificativa e a sequência de passos proposta para a migração gradual de microsserviços. O objetivo é fornecer um guia prático, com comandos e checklists, para cada etapa do processo de refatoração, visando uma transição controlada e bem-sucedida.

O sumário executivo abaixo apresenta um roteiro estratégico, explicando por que cada fase é crucial para minimizar riscos e maximizar a eficiência da migração.



---

## Provedor Mínimo (PoC)

**Justificativa:** Iniciar com um alvo simples e controlado permite implementar rapidamente um contrato mínimo, validando o conceito de "service" sem a complexidade total do framework do monólito. Isso demonstra a viabilidade da abordagem e estabelece uma base sólida.

**Resultado esperado:** Um servidor pequeno (utilizando **Express** ou Nest.js em sua forma mais básica) que responde aos endpoints necessários com respostas mockadas, comprovando a comunicação e o contrato.

## Instruções para o Provedor Mínimo (PoC) do Serviço de Usuários

### Prompt

Crie uma PoC de "provider minimal" para o serviço de users (Express) dentro deste monorepo em users-service. Requisitos:

Ambiente: Node 18+, preferência por TypeScript (se o pacote já usa TS, mantenha TS).

Local: users-service (não toque no monolito users).

Contract source: use openapi.yaml como referência. Se versionar a spec, usar openapi: 3.1.0.

Endpoints a implementar (mocked):

- GET /health  
Retorno 200: { status: "ok" }
- POST /users  
Body: { email: string, name?: string, password?: string } (password opcional para PoC)  
Cenários:
  - Dados válidos -> 201 { id: string, email: string, name?: string }
  - Dados inválidos -> 400 { message: "Invalid payload" }
- GET /users/:id  
Retorno 200 { id: string, email: string, name?: string } ou 404 { message: "User not found" }

## Implementação:

- Servidor Express básico com `express.json()` e `cors()`.
- Respostas mockadas (sem persistência). IDs e demais campos podem ser strings estáticas (ex.: "mocked-user-id") ou geradas com `uuid/v4`.
- Exportar a app (`module.exports / export default app`) para que testes com `supertest` possam importar `app`.
- Entrypoint `src/server.ts` (ou `src/index.ts`) que chama `app.listen` para `npm run dev`.

Este documento detalha o prompt adaptado para a criação de uma Prova de Conceito (PoC) do serviço de usuários, utilizando o framework `Express.js`. O objetivo é estabelecer um "provedor mínimo" que valide a comunicação e o contrato de serviço em um ambiente isolado, sem a complexidade total do monólito existente.

A implementação deste PoC permitirá demonstrar a viabilidade da abordagem de microsserviços para o domínio de usuários, servindo como base para futuras extrações e refatorações.

## Requisitos Técnicos e Estrutura do Projeto

- **Ambiente:** Node 18+ com preferência para TypeScript (se o pacote já utiliza TS, manter a linguagem).
- **Localização:** O código do PoC deve residir em um novo diretório, `users-service`, dentro da estrutura do monorepo, sem interferir no código existente do monólito.
- **Fonte do Contrato:** Utilizar o arquivo `openapi.yaml` como referência principal para a definição dos endpoints e estruturas de dados. Caso a especificação seja versionada, utilizar `openapi: 3.1.0`.

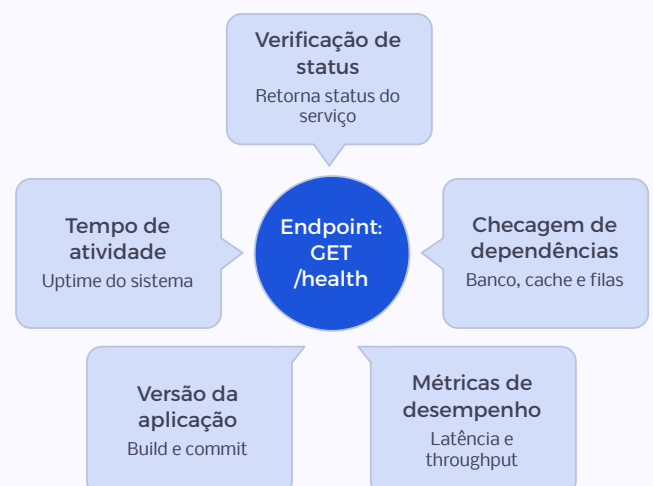
## Endpoints a Implementar (Mockados)

Os seguintes endpoints devem ser implementados com respostas mockadas para validar o contrato de API e a comunicação básica:

### GET /health

Um endpoint simples para verificar a saúde do serviço.

- **Retorno esperado:** 200 OK com o corpo `{ "status": "ok" }`



## POST /users

Endpoint para a criação de novos usuários, com validação básica de payload.

- **Body esperado:** { "email": string, "name"?: string, "password"?: string } (o campo password é opcional para esta PoC).
- **Cenário 1 (Dados válidos):** Retorno 201 Created com o corpo { "id": string, "email": string, "name"?: string }.
- **Cenário 2 (Dados inválidos):** Retorno 400 Bad Request com o corpo { "message": "Invalid payload" }.

## GET /users/:id

Endpoint para a recuperação de um usuário específico por ID.

- **Cenário 1 (Usuário encontrado):** Retorno 200 OK com o corpo { "id": string, "email": string, "name"?: string }.
- **Cenário 2 (Usuário não encontrado):** Retorno 404 Not Found com o corpo { "message": "User not found" }.

# Implementação do Provedor Mínimo (PoC)

Para garantir a validação eficiente do contrato e da comunicação básica em um ambiente controlado, o provedor mínimo do serviço de usuários deve aderir às seguintes especificações técnicas de implementação. Estas diretrizes visam simplificar o desenvolvimento da prova de conceito, focando na funcionalidade essencial e na integração com ferramentas de teste.

- Configuração de um servidor **Express** básico, incluindo os middlewares `express.json()` para parsing de JSON e `cors()` para gerenciamento de políticas de segurança entre origens, garantindo a interoperabilidade com clientes.
- Implementação de respostas totalmente **mockadas**, sem a necessidade de persistência em banco de dados. IDs e outros campos dinâmicos podem ser representados por strings estáticas (ex: "mocked-user-id") ou gerados programaticamente utilizando bibliotecas como `uuid/v4` para simular dados únicos.
- A instância principal da aplicação Express (app) deve ser **exportada** (via `module.exports` ou `export default app`). Esta prática é fundamental para permitir que ferramentas de teste, como `supertest`, importem a aplicação diretamente, facilitando a escrita de testes de integração e de contrato.

- O ponto de entrada do serviço (`src/server.ts` ou `src/index.ts`) deve ser responsável por iniciar a aplicação Express, chamando `app.listen()` para que o servidor possa ser executado e escutar requisições, tipicamente através do comando `npm run dev`.

# OpenAPI

## Entendendo o Contrato de Serviço

O arquivo `packages/users-service/openapi.yaml` é um documento crucial que serve como o contrato formal para o nosso serviço de usuários. Baseado na especificação OpenAPI 3.1.0, ele descreve de forma clara e legível todas as capacidades e a interface de comunicação do serviço, mesmo para esta Prova de Conceito (PoC).

Sua existência é fundamental para garantir a **interoperabilidade**, facilitar o desenvolvimento colaborativo e automatizar processos importantes, como a geração de documentação e clientes de API. Ele é o ponto de verdade sobre como outros sistemas podem interagir com o `users-service`.

## Estrutura e Componentes Principais do `openapi.yaml`

A especificação OpenAPI organiza a descrição da API em seções lógicas, fornecendo uma visão completa de seus endpoints e modelos de dados. Para o `users-service`, os principais elementos incluem:

- **Info:** Contém metadados básicos sobre a API, como título (`Users Service (PoC)`) e versão (`0.1.0`).
- **Paths:** Descreve os caminhos (endpoints) da API e as operações HTTP associadas (GET, POST, etc.).
- **Components/Schemas:** Define os modelos de dados (DTOs) que são utilizados nas requisições e respostas da API, como `CreateUser` e `User`.

### Endpoints Mínimos Definidos

- `GET /health`: Um endpoint simples para verificar a saúde do serviço, retornando `200 OK`.
- `POST /users`: Utilizado para criar novos usuários. Espera um corpo de requisição do tipo `CreateUser` e retorna um objeto `User` com status `201 Created`.
- `GET /users/{id}`: Permite buscar um usuário específico por seu ID. Retorna um objeto `User` com status `200 OK` ou `404 Not Found` caso o usuário não exista.

## Modelos de Dados (Schemas)

- `CreateUser`: Define a estrutura dos dados para a criação de um novo usuário. Inclui `email` (obrigatório), `name` e `password` (opcionais para a PoC).
- `User`: Representa um usuário registrado no sistema. Contém os campos `id`, `email` e `name`.

# A Importância da Especificação para o Desenvolvimento

Embora o escopo inicial seja uma PoC enxuta, o `openapi.yaml` já traz benefícios significativos para o ciclo de desenvolvimento:

## Guia para Implementação

A especificação serve como um blueprint para o desenvolvimento dos controladores (no caso do Express, as rotas e seus handlers) e dos objetos de transferência de dados (DTOs), garantindo que a implementação esteja alinhada com o contrato definido.

## Base para Testes de Contrato

Com a especificação, é possível criar testes de contrato automatizados que verificam se a implementação da API realmente atende ao que foi prometido no `openapi.yaml`, prevenindo quebras em integrações.

# O que é OpenAPI?

OpenAPI é um padrão poderoso e independente de framework para descrever APIs RESTful de forma padronizada. Ele fornece uma linguagem comum para que tanto humanos quanto máquinas possam entender as capacidades de uma API sem a necessidade de acesso ao código-fonte ou documentação adicional. Essencialmente, é um contrato legível por máquina para suas APIs.

- Define os endpoints (paths) disponíveis e os métodos HTTP associados (GET, POST, PUT, DELETE, etc.).
- Detalha os parâmetros que podem ser enviados (query, path, header, body), seus tipos e validações.
- Especifica os formatos esperados para requisições e respostas (content types) e os esquemas (models) dos dados.
- Pode incluir informações adicionais como segurança, tags para organização e exemplos de uso.
- Não está vinculado a uma linguagem de programação ou framework específico, podendo ser utilizado com qualquer tecnologia de backend, como Express, Nest.js, Spring Boot, Flask, etc.

# Abordagens Comuns para Integração com OpenAPI

Existem duas metodologias principais para integrar a especificação OpenAPI no ciclo de vida de desenvolvimento de APIs, cada uma com seus próprios benefícios e casos de uso, dependendo da prioridade do projeto.

## Code-First

Nesta abordagem, o código da API é a fonte primária de verdade. A especificação OpenAPI é gerada automaticamente a partir de anotações no código-fonte (por exemplo, usando bibliotecas como `@nestjs/swagger` no NestJS ou Swashbuckle no .NET). Isso garante que a documentação esteja sempre sincronizada com a implementação, mas o design da API é ditado pelo código.

## Contract-First

Aqui, o documento `openapi.yaml` (ou `.json`) é a fonte da verdade e é criado antes de qualquer implementação de código. A partir dele, são gerados clientes, stubs de servidor e modelos de dados para diferentes linguagens. Essa abordagem promove um design de API robusto e colaborativo, onde o contrato é definido e acordado antes do desenvolvimento.

No nosso repositório, observamos uma abordagem híbrida: enquanto a aplicação principal utiliza `@nestjs/swagger` para gerar documentação em tempo de execução (code-first), cada pacote de microsserviço mantém um arquivo `openapi.yaml` estático, funcionando como um contrato para fins de PoC e testes.

## Por que Manter um `openapi.yaml` Estático com `@nestjs/swagger`?

Mesmo com a capacidade de gerar a especificação OpenAPI dinamicamente através de `@nestjs/swagger`, o projeto opta por manter arquivos `openapi.yaml` estáticos dentro de cada pacote de serviço. Essa decisão estratégica oferece múltiplas vantagens:

### Testes de Contrato Robustos

A existência do `openapi.yaml` permite a criação de testes de contrato automatizados. Esses testes verificam se a implementação da API (o código real) adere estritamente ao que foi definido no contrato. Isso previne regressões e garante a estabilidade das integrações entre serviços, mesmo com evoluções independentes.

### Material Didático e PoC

Os arquivos `openapi.yaml` atuam como um guia estático e autoexplicativo, servindo como material didático e exemplos claros para novos desenvolvedores ou para propósitos de Prova de Conceito (PoC). Eles facilitam o entendimento rápido da API sem a necessidade de configurar o ambiente de desenvolvimento completo.

## Geração Simplificada de Clientes e Stubs

Um contrato estático e bem definido é ideal para a geração automática de SDKs (kits de desenvolvimento de software) ou stubs de cliente em diversas linguagens. Isso acelera o desenvolvimento em sistemas consumidores da API, reduzindo o esforço manual e minimizando erros de integração.

## Documentação Versionada e Consistente

Manter o `openapi.yaml` diretamente no repositório permite versionar a documentação da API junto com o código, garantindo que as especificações estejam sempre alinhadas com as versões correspondentes do serviço. Isso promove uma fonte única de verdade para a API.

# Mapeando OpenAPI para Express: Uma Abordagem Prática

Para desenvolvedores que buscam integrar a especificação OpenAPI (anteriormente Swagger) em suas aplicações Express.js, é fundamental compreender como cada elemento do contrato YAML se traduz em componentes de código. Esta seção detalha essa correspondência, oferecendo um guia claro para a implementação e o ensino de APIs RESTful.



## Paths (Rotas)

Cada `path` (caminho) definido no `openapi.yaml` corresponde diretamente a uma rota específica no Express. Por exemplo, um `GET /users/{id}` é implementado com `app.get('/users/{id}', handler)` ou `router.get('/:id', handler)`, onde o handler é a função que processa a requisição.



## Request Body e Schemas

Os schemas que descrevem o corpo das requisições, como o `CreateUser`, são representados no Express por interfaces TypeScript ou classes de DTOs (Data Transfer Objects). A validação desses dados é crucial e pode ser feita utilizando bibliotecas como `express-validator`, `AJV` ou `Joi` como middlewares na rota.



## Responses (Respostas)

A estrutura das respostas esperadas no OpenAPI é espelhada na forma como o Express envia dados de volta ao cliente. Isso geralmente é feito através de `res.status(...).json(...)`, garantindo que o formato e o status HTTP da resposta estejam de acordo com a especificação.



## Parameters (Parâmetros)

Os parâmetros de requisição (`query`, `path`, `header`) são facilmente acessíveis nos objetos `req.params`, `req.query` e `req.headers` do Express, respectivamente. Isso permite que a lógica da aplicação utilize as informações passadas pelo cliente conforme definido no OpenAPI.



## securitySchemes (Segurança)

As definições de segurança no `openapi.yaml` (ex: autenticação via JWT) são implementadas no Express como `middlewares`. Um `middleware` de autenticação, por exemplo, verificaria o `token` e chamaria `next()` se a autenticação for válida, permitindo que a requisição prossiga para o `handler` da rota.

Entender essas correspondências é o alicerce para construir APIs Express robustas e bem documentadas, facilitando não apenas o desenvolvimento, mas também a manutenção e a integração com outros serviços e clientes.

# Exemplos Práticos: Rotas e Validação em Express

Para solidificar a compreensão da integração entre a especificação OpenAPI e a implementação em Express.js, apresentamos um exemplo mínimo de um arquivo de rotas. Este trecho de código demonstra a criação de endpoints RESTful, a manipulação de parâmetros de rota, o corpo da requisição e, crucialmente, a validação de entrada usando `express-validator`, um `middleware` popular para Express.

Este exemplo é ideal como material didático, pois ilustra de forma concisa como os conceitos de `Paths`, `Parameters`, `Request Body` e `Responses` definidos no OpenAPI se traduzem em código funcional e seguro.

```
// users.routes.js
const express = require('express');
const { body, param, validationResult } = require('express-validator');

const router = express.Router();

router.get('/:id',
  param('id').isString().notEmpty(),
  (req, res) => {
    const errors = validationResult(req);
```



```
if (!errors.isEmpty())
  return res.status(400).json({ errors: errors.array() });
const { id } = req.params;
// buscar usuário (mock)
res.json({ id, email: 'user@example.com', name: 'User' });
}
);

router.post('/',
  body('email').isEmail(),
  body('name').optional().isString(),
  body('password').optional().isString(),
  (req, res) => {
    const errors = validationResult(req);
    if (!errors.isEmpty())
      return res.status(400).json({ errors: errors.array() });
    const { email, name } = req.body;
    // criar usuário (mock)
    res.status(201).json({ id: '123', email, name });
  }
);

module.exports = router;
```

Analisando o código acima, podemos destacar os seguintes pontos essenciais para a construção de APIs robustas com Express e OpenAPI:

### Definição de Rotas

As chamadas `router.get('/:id', ...)` e `router.post('/', ...)` estabelecem os endpoints da API, correspondendo diretamente aos `Paths` e métodos HTTP especificados no OpenAPI. Cada rota define o tipo de requisição e o caminho URL.

### Validação de Entrada

A biblioteca `express-validator` é utilizada para garantir que os dados recebidos (sejam de `param` ou `body`) estejam em conformidade com o `Schema` definido no OpenAPI. Se a validação falhar, um erro 400 é retornado, prevenindo o processamento de dados inválidos.

### Acesso aos Dados da Requisição

O Express facilita o acesso aos dados da requisição via `req.params` (para parâmetros de rota como `:id`) e `req.body` (para o corpo da requisição JSON em requisições `POST`). Isso permite que a lógica da aplicação utilize as informações conforme esperado.

## Respostas da API

A função `res.json(...)` envia uma resposta JSON ao cliente, enquanto `res.status(...)` define o código de status HTTP (e.g., 200 OK, 201 Created, 400 Bad Request), assegurando que o contrato de Responses do OpenAPI seja atendido tanto no formato quanto no status.

Este modelo fornece uma base clara para desenvolver e manter APIs que são tanto funcionais quanto bem documentadas, alinhadas com as melhores práticas de design de API e contrato OpenAPI.

# Validação em Runtime com `express-openapi-validator`

Como alternativa à validação manual utilizando bibliotecas como `express-validator`, é possível automatizar grande parte desse processo integrando o serviço Express diretamente com o contrato `openapi.yaml`. A biblioteca `express-openapi-validator` oferece uma solução poderosa para validar requisições e respostas automaticamente em runtime, garantindo que o tráfego da API esteja sempre em conformidade com a especificação.

Esta abordagem reduz a quantidade de código boilerplate e centraliza a fonte da verdade para a validação na própria especificação OpenAPI, facilitando a manutenção e garantindo a consistência entre a documentação e a implementação.

```
// app.js ou server.ts
const path = require('path');
const express = require('express');
const OpenApiValidator = require('express-openapi-validator');

const app = express();
app.use(express.json()); // Habilita o parsing de JSON no corpo da requisição

app.use(
  OpenApiValidator.middleware({
    apiSpec: path.join(__dirname, 'packages/users-service/openapi.yaml'),
    validateRequests: true, // Valida automaticamente os corpos de
      // requisição e parâmetros
    validateResponses: false, // Opcional: define se as respostas também
      // devem ser validadas
  })
);
```

```
// As rotas do Express são registradas normalmente aqui.  
// Os handlers devem estar de acordo com a especificação OpenAPI,  
// pois o validador já garantiu a conformidade da entrada.  
  
app.listen(3010, () => {  
  console.log('Servidor rodando na porta 3010');  
});
```

Ao integrar o `express-openapi-validator` como um middleware global, todas as requisições recebidas serão automaticamente verificadas contra os `schemas` definidos no arquivo `openapi.yaml`. Se uma requisição não estiver em conformidade com o contrato (por exemplo, um campo obrigatório faltando ou um tipo de dado incorreto), o validador interceptará a requisição e retornará um erro HTTP 400 (Bad Request) antes mesmo que a lógica do seu handler seja executada. Isso promove uma API mais robusta e segura.

## Execução e Testes Rápidos do Serviço

Após a configuração do serviço com a validação OpenAPI em runtime, é fundamental garantir que o ambiente esteja pronto para execução e que os endpoints básicos funcionem conforme o esperado. Esta seção detalha os passos para inicializar o serviço, instalar as dependências necessárias e realizar testes rápidos utilizando comandos `curl` para verificar a conectividade e a funcionalidade básica dos endpoints.

### Inicialização e Instalação de Dependências

Para começar, navegue até o diretório do serviço de usuários e instale as dependências. Estes passos garantem que todas as bibliotecas necessárias, como Express, `express-validator` e `express-openapi-validator`, estejam disponíveis.

```
cd packages/users-service  
npm init -y  
npm install express express-validator express-openapi-validator yaml
```

### Execução do Servidor

Com as dependências instaladas, o serviço pode ser iniciado. O comando abaixo executa o arquivo `server.js` (ou `app.js`, dependendo da sua configuração), que, conforme visto anteriormente, contém a lógica para iniciar o servidor Express e escutar as requisições na porta configurada (neste caso, 3010).

Ainda no diretório users-service:

```
node server.js
```

Saída esperada

```
users-service listening on http://localhost:3010
```

## Testes Rápidos com `curl`

Para validar a comunicação com o serviço e a correta resposta dos endpoints, utilize os seguintes comandos `curl`. Eles simulam requisições HTTP para verificar o endpoint de saúde, a criação de um novo usuário e a consulta de um usuário existente.

### Verificar Saúde do Serviço

Confirma se o servidor está ativo e respondendo.

```
curl http://localhost:3010/health
```



### Criar Novo Usuário

Testa o endpoint `POST /users` para a criação de um usuário com dados válidos.

```
curl -X POST http://localhost:3010/users -H 'Content-Type: application/json' -d '{"email":"aluno@example.com","name":"Aluno"}'
```



### Consultar Usuário por ID

Verifica o endpoint `GET /users/{id}` para a recuperação de um usuário mockado.

```
curl http://localhost:3010/users/123
```

Esses testes básicos são cruciais para confirmar que o serviço está operacional e que a camada de validação OpenAPI está funcionando corretamente, rejeitando requisições inválidas e processando as válidas. Em um ambiente de desenvolvimento real, esses seriam os primeiros passos para garantir a estabilidade do microserviço recém-criado.